IOTA JU Redesign
Shreyam: Source Inventory, Migration and Media

Implementation handout

21 October 2026

1 Your job

You own the source inventory, safe repeatable content import, media storage/delivery, media components and final parity evidence. Your job is to make sure the redesigned site contains the right source content and that every photograph, report, embed and video can be used safely and efficiently.

Plain language: an inventory is a list of every source page, asset and old URL. A manifest is the machine-readable version of that list. An importer copies approved source content into Payload records. A checksum is a fingerprint used to detect whether a file changed. A rendition is a smaller delivery copy; it never replaces the original.

2 Install and access

Install Git, the supported Node.js LTS version (at least 20.18.1 for the current Payload requirement) and pnpm, then:

```
pnpm install --frozen-lockfile
```

Install FFmpeg and FFprobe on your workstation for video inspection/preparation. Confirm both commands work with ffmpeg -version and ffprobe -version. Use the project's existing sharp dependency for images; ask Sourasish before adding root packages. Keep original source assets in the approved source snapshot or object storage, not in Git.

You need a read-only snapshot of the legacy website and approval to use the staging database/storage. Ask the orchestrator for access. Do not assume the old checkout path from an earlier computer session exists. Never use production credentials during development.

3 Your folders and files

Files/folders you own	Purpose
`data/migration/source-manifest.json`	Stable list of content, files, old URLs and migration decisions; store metadata, not large media binaries.
`docs/source-inventory.md`	Human-readable inventory, unknowns and owner-review list.

src/collections/Media.ts	Media record fields and access rules; coordinate collection registration with Ritendu.
src/lib/media/	Narrow storage/upload helpers for the provider selected by the orchestrator.
scripts/content/inventory.ts, scripts/content/verify.ts	Repeatable source inventory and public parity commands.
scripts/import/	Dry-run-first mapping and import scripts.
scripts/media/prepare.ts	Repeatable image/video delivery preparation and checksum manifest.
src/components/media/	Shared image, background-video, demo/embed and report-link components used by Pritish.
tests/media-access.test.ts, tests/import.test.ts, tests/e2e/media.spec.ts	Secure media, repeat-import and real-browser playback checks.
docs/content-migration-runbook.md	Final reconciliation and editor/import handover.

Do not edit root dependencies, shared Payload config or public page layouts without coordination. Sourasish owns root configuration; Ritendu owns all collections except Media; Pritish owns page structure and styling.

4 M01: inventory the exact source

Use the latest source revision supplied by the orchestrator. Record its commit or export date. Earlier counts are only clues; rerun inventory against the current content before migration.

Create these files:

- scripts/content/inventory.ts
- data/migration/source-manifest.json
- docs/source-inventory.md

Include every public page/detail page, exact old URL and casing, visible text, image/video/PDF, external embed and inbound link. For each item mark migrate, redirect, retain or exclude, with a reason and a person who must resolve uncertainty.

For each media file record original path, filename, exact casing/encoding, MIME/type, byte size, SHA-256, duration/dimensions where applicable, references and privacy decision. Do not delete or merge files based only on names or equal sizes. Record source discrepancies, duplicate people, questionable dates/statuses, stale event actions, unavailable links and missing files as review items.

The new app does not include the workshop Q&A backend. Record its exclusion and the need for an owner-approved old-URL response; do not delete the legacy service or data.

Check: run inventory twice on the same source revision; outputs should be deterministic. Trace a project, event, person, home video, PDF and linked historical detail from source to manifest.

5 M02: safe media records and uploads

After the orchestrator approves a storage provider and budget, define Media with Ritendu. Create src/collections/Media.ts and the smallest required provider adapter under src/lib/media/. Coordinate

database fields/registration with Ritendu and environment names with Sourasish.

Keep private originals/draft uploads distinct from approved public delivery files. Use short-lived direct-to-storage upload permissions for large files; do not send a full video through a Vercel function. The current Vercel Function body limit is 4.5 MB. Validate file type, size, scope and upload completion before a record can be attached or published. Expired, anonymous, wrong-scope and incomplete uploads must fail safely.

Check: `pnpmtest--tests/media-access.test.ts`. Try signed out, expired permission, mismatched file metadata, interrupted upload, allowed public file and protected private master. Verify the storage URL itself cannot leak a draft.

6 M03: prepare delivery copies and keep originals

Create `scripts/media/prepare.ts`, a checksum/rendition manifest and a brief quality report. Use FFmpeg/FFprobe for supported video conversions and inspection; use sharp already installed in the app for image variants. Never overwrite a master. Do not commit large media files into this repository.

Prepare responsive image variants and posters. For the homepage background, try an 8–15 second loop around 2–4 MB only where image quality remains acceptable. Keep the complete source demo video and its duration/orientation for users who open it. Keep report documents downloadable. If a codec/source is unsupported, record it and request a decision rather than dropping it.

Check: hash originals before/after; run the preparation twice; confirm there are no duplicate output files; compare poster and image quality; play one actual delivery video in a browser. Do not build an automatic request-time transcoding service.

7 M04: import one real project safely

After Ritendu's Project contract and Sourasish's isolated staging environment are ready, build the import core in `scripts/import/`. Start with one real TemperLink record, its verified copy and a working video/report reference.

The default command must only preview changes:

```
pnpm content:import -- \\
  --manifest data/migration/source-manifest.json \\
  --group temperlink \\
  --dry-run
```

Writes require both an explicit apply flag and named target environment. Production must be refused outside the approved release procedure. Use stable source-kind/legacy keys for repeat runs. Report conflicts; do not silently replace an editor's later CMS changes. Partial failure must be resumable without duplicate people/media/projects.

Check: `pnpmtest--tests/import.test.ts`. Test dry-run, two identical applies on isolated staging, a changed source file, conflict with an edited CMS record and a partial failure. Demonstrate the imported project in Pritish's list/detail/home teaser and test its media.

8 M05a: remaining projects and AGV history

Extend the same importer using your manifest and Ritendu's model. Preserve the current 11-card source ordering, exact approved copy/status/award labels and all working reports/videos. Account for the separate

AGV feature and linked/unlisted historical detail pages.

Do not invent contributors, merge project records or hide a project just because it is not a main card. Re-run imports without overwriting approved CMS edits; report conflicts for resolution.

Check: compare each source manifest entry with the CMS record, order, detail path, media and report. Any unresolved mismatch must be named and assigned.

9 M05b: events and historical pages

Extend the importer for event pages and historical detail content. Preserve original text, posters, galleries, PDFs, external embeds and known date precision. Do not restore expired Register Now/Live Portal actions. Unknown dates and questionable current states go to the owner-review list.

Check: compare one current and two historic event formats from source through imported CMS content to the rendered page. Open the real report/PDF and play a full demo, not merely an HTTP check.

10 M05c: people, memberships and alumni

Extend import mapping for People and Memberships after Ritendu's identity review. Confirmed repeated people become one person with all correct group/year appearances. Same-name entries remain separate until verified. Preserve actual departments, roles, order, portraits and approved public links; never guess unknown values.

Preserve the corrected Electrical Engineering department for Raunak Panja and Ritendu's approved department text. Build a reconciliation list of source appearances and CMS memberships.

Check: use fixtures for same-name different people and one person in multiple groups. Compare source appearance counts to CMS memberships and inspect the team/alumni pages with Pritish.

11 M05d: home, About, timeline and redirects

Map remaining home/About/settings/timeline data and exact old paths. Store selected media references rather than copying a parallel content source into the UI. Keep official contact address `iotajuofficial@gmail.com`. Link featured content only to records that are safe to publish.

For every old public URL, record a destination/retention decision for Sourasish's redirect runtime. Keep Q&A retirement separate; the orchestrator must approve the public response.

Check: compare source text and media links with records, check references, then run the legacy-route tests with Sourasish.

12 M06: shared media components

Create reusable components under `src/components/media/`, agreeing the inputs with Pritish before implementation. A component receives the CMS media record and a purpose such as background, demo, image or report; it must not assume every file is an MP4.

- Background video: immediate poster, muted inline playback when allowed, accessible pause/play, text overlay readability, reduced-motion/data-saving fallback, pause when off screen.
- Demo: real controls, lazy loading, no full video download on the project catalogue, safe approved embeds.

- Images/portraits: responsive size, reserved dimensions, correct crop and useful alt text.
- Reports: clear link and file type/size where available.

Check: `pnpmtest:e2e--tests/e2e/media.spec.ts`. Use a real phone or device emulation; test keyboard controls, autoplay failure, reduced motion, scrolling offscreen, network requests and failed/missing media.

13 M07: whole-site parity

Create `scripts/content/verify.ts` and a clear report of migrated, redirected, retained, excluded, missing and unresolved entries. Do not count an intentionally excluded old portal as a missing public page. Check exact casing/encoding and final links. Verify a video actually plays and a PDF downloads.

Check:

```
pnpm content:verify -- \\
  --manifest data/migration/source-manifest.json \\
  --base-url <staging-url>
```

Run against merged staging, then deliberately break a test link and confirm the verifier reports it. Unexpected private media exposure, missing reports, unresolved conflicts or unreviewed text/ordering differences block release.

14 M08: final reconciliation and handover

Create `docs/content-migration-runbook.md`. Reconcile source changes made while the new site was built against approved CMS edits; never silently pick one side. Document import order, backup prerequisite, dry-run, apply, conflict handling, resume, verification and rollback.

Rehearse a second source revision plus one CMS edit conflict in staging. At launch, perform only the manifest/environment procedure approved by the orchestrator. Preserve every original and the legacy site.

15 Handoffs and rules

- Ritendu owns record schemas and migrations; agree Media fields before using them in import scripts.
- Sourasish owns root dependencies, storage environment naming and deployment configuration. Do not change these alone.
- Pritish owns page layout. Agree the media component API and design tokens with him before styling inside shared media components.
- The orchestrator approves content ambiguities, provider spending, production imports and final cutover.
- Import is dry-run by default; use explicit named environment for every write. Never point tests at production.
- Keep originals outside Git; use checksums and references before suggesting any cleanup.

16 Official references

See the official Payload upload storage adapters and Payload collection access controls. Current Vercel Function limits document the 4.5 MB body limit. Re-check the official provider documentation after the orchestrator selects object storage. Checked 21 October 2026.